Exercise Sheet 04

The Staff List

House rules.

- One file per exercise, in `my_work/`: `ex1_staff_list.py`, `ex2_salaries.py`, …
- Type the **Given** lines first. Then write the rest yourself. Try each step in the REPL before you put it in the file.
- Run it (`python my_work/ex1_staff_list.py`) and compare with the **expected output**, character by character. When Python prints a list or a dictionary itself, it uses single quotes. That's fine; don't fight it.
- Hand in: commit and push to `lesson-04`, then open a pull request. Tell Lars one thing a dictionary does better than a list.

Exercise 1 The staff list

10 min · `ex1_staff_list.py`

> **From Priya (Payroll):** *"Mo says you've been keeping staff in a text with slashes in it. He said it in the tone he uses for people who store dates as text. Apparently there's a proper thing."*

GIVEN

```
staff = ["Priya Patel", "Mo Khan", "Siobhan Walsh", "Tom Brown"]
```

(a) Print the first person, the last person, and how many there are.

(b) Amara Okafor has joined. `append` her and print the whole list.

(c) Print whether `"Mo Khan"` is in the list, and whether `"Dave"` is.

(d) Tom Brown has left. `remove` him and print the list again.

(e) Print a slice of the second and third people. Slicing works like it did on text.

EXPECTED OUTPUT

```
Priya Patel
Tom Brown
4
['Priya Patel', 'Mo Khan', 'Siobhan Walsh', 'Tom Brown', 'Amara Okafor']
True
False
['Priya Patel', 'Mo Khan', 'Siobhan Walsh', 'Amara Okafor']
['Mo Khan', 'Siobhan Walsh']
```

Hint: Position 0 is the first, −1 is the last. Same rule as for letters in a string: a list is a row of things, a string is a row of characters.

Exercise 2 A column of numbers

10 min · `ex2_salaries.py`

> **From Gerald (MD):** *"SUM, MAX, MIN, AVERAGE, and the top three. The things I do in Excel while pretending to listen on calls. G."*

GIVEN

```
salaries = [31000, 36500, 28000, 24500, 41000, 52000, 19500, 33000]
```

(a) Print the total, then the highest and lowest on one line, then the average rounded to 2 decimals.

(b) Print the salaries sorted low to high, then the top three (sort the other way, slice the first three).

(c) Build a new list `above` of the salaries over 30k: start with `[]`, loop, `append`. Print it, then "5 of 8 earn over 30k" using `len` twice.

EXPECTED OUTPUT

```
265500
52000 19500
33187.5
[19500, 24500, 28000, 31000, 33000, 36500, 41000, 52000]
[52000, 41000, 36500]
[31000, 36500, 41000, 52000, 33000]
5 of 8 earn over 30k
```

Hint: `sorted(salaries)` gives a sorted copy and leaves the original alone. `salaries.sort()` sorts in place and returns nothing. Guess which one Dave used in a print.

Exercise 3 One employee, many fields

10 min · `ex3_employee_dict.py`

> **From Mo (IT):** *"A person isn't a list. A person has a name, a department, a salary. In Python that's a dictionary: labels on the left, values on the right. Like one row of a table, with the headers attached."*

GIVEN

```
employee = {
    "name": "Priya Patel",
    "department": "Payroll",
    "salary": 31000,
    "hours": 37.5,
}
```

(a) Print the name, then the monthly salary (just divide; no rounding this time).

(b) Add a new field `"rating"` with the value 4, change the salary to 32000, and print the whole dictionary.

(c) Print `employee.get("bonus", 0)`. There is no bonus field: `.get` gives you the fallback instead of crashing. Then print how many fields there are.

(d) Loop over the dictionary (`for key in employee:`) and print each key left-aligned in 12 characters followed by its value.

EXPECTED OUTPUT

```
Priya Patel
2583.3333333333335
{'name': 'Priya Patel', 'department': 'Payroll', 'salary': 32000, 'hours': 37.5, 'rating':
4}
0
5
name        Priya Patel
department  Payroll
salary      32000
hours       37.5
rating      4
```

Exercise 4 A list of dictionaries

15 min · `ex4_staff_records.py`

> **From Priya:** *"So a dictionary is a row and a list is a column. Can I have a table? With people in it? Who each have a salary?"*

GIVEN

```
staff = [
    {"name": "Priya Patel", "dept": "Payroll", "salary": 31000, "gender": "F"},
    {"name": "Mo Khan", "dept": "IT", "salary": 36500, "gender": "M"},
    {"name": "Siobhan Walsh", "dept": "Reward", "salary": 28000, "gender": "F"},
    {"name": "Tom Brown", "dept": "Warehouse", "salary": 24500, "gender": "M"},
    {"name": "Amara Okafor", "dept": "Sales", "salary": 41000, "gender": "F"},
]
```

(a) Loop over the list: each `person` is a dictionary. Print one line each: name in 15, department in 11, then the salary as `£31,000` right-aligned in 8. (Build the money text first, then align it.)

(b) Add up the salaries into a total and print it.

(c) Find the highest-paid person. Start with `top = staff[0]`, loop, and replace `top` whenever someone earns more. Print their name and department.

(d) Build a list of the women's names and print it.

EXPECTED OUTPUT

```
Priya Patel    Payroll     £31,000
Mo Khan        IT          £36,500
Siobhan Walsh  Reward      £28,000
Tom Brown      Warehouse   £24,500
Amara Okafor   Sales       £41,000
Total payroll: £161,000
Highest paid: Amara Okafor (Sales)
['Priya Patel', 'Siobhan Walsh', 'Amara Okafor']
```

Hint: Inside an f-string, use the other kind of quotes for the key: `f"{person['name']}"`.

Exercise 5 Counting with a dictionary

10 min · `ex5_headcount.py`

> **From Siobhan (Reward):** *"Headcount per department. In Excel it's a COUNTIF per department, and someone always forgets the new department. I want it to notice new departments by itself."*

GIVEN

```
departments = ["Sales", "IT", "Payroll", "Sales", "Warehouse", "IT", "Sales", "HR", "IT",
"Sales"]
```

(a) Start with an empty dictionary `headcount = {}`. Loop over the departments: if the department is already a key, add 1; otherwise create it with 1. Print the dictionary.

(b) Print the counts as a table, departments in alphabetical order (`sorted(headcount)` sorts the keys).

(c) Find the biggest department and print it with its count.

EXPECTED OUTPUT

```
{'Sales': 4, 'IT': 3, 'Payroll': 1, 'Warehouse': 1, 'HR': 1}
HR          1
IT          3
Payroll     1
Sales       4
Warehouse   1
Biggest department: Sales (4 people)
```

Hint: This "if it's there add, otherwise create" loop is the most common five lines in all of data work. You'll write it a hundred times, until pandas does it for you in one.

Exercise 6 XLOOKUP, the Python way

10 min · `ex6_lookup.py`

> **From Priya:** *"Half my spreadsheets are XLOOKUPs: band to minimum salary, department to manager. Tell me a dictionary is an XLOOKUP and I'll stop being scared of it."*

GIVEN

```
band_minimum = {"A": 0, "B": 25000, "C": 32000, "D": 40000}
dept_manager = {"Sales": "Amara Okafor", "IT": "Mo Khan", "Payroll": "Priya Patel"}
```

(a) Look up band C's minimum and IT's manager. Then look up HR's manager with `.get` and the fallback `"no manager yet"`. Then print whether `"Payroll"` is a key.

(b) Print the keys of `dept_manager` as a list, and the values of `band_minimum` as a list.

(c) Loop with `for band, minimum in band_minimum.items():` and print one line per band.

(d) Given `salary = 33000`, work out its band: loop over the items, and whenever the salary is at least the minimum, remember that band. Print the result.

EXPECTED OUTPUT

```
32000
Mo Khan
no manager yet
True
['Sales', 'IT', 'Payroll']
[0, 25000, 32000, 40000]
Band A starts at £0
Band B starts at £25,000
Band C starts at £32,000
Band D starts at £40,000
£33,000 is in band C
```

Exercise 7 The pay gap, by department STRETCH

15 min · `ex7_pay_gap_by_dept.py`

> **From Gerald:** *"Pay gap per department. For the board. By Thursday. (Which Thursday? A Thursday.) G."*

GIVEN

```
staff = [
    {"name": "Priya Patel", "dept": "Payroll", "hourly": 15.90, "gender": "F"},
    {"name": "Mo Khan", "dept": "IT", "hourly": 18.72, "gender": "M"},
    {"name": "Siobhan Walsh", "dept": "Payroll", "hourly": 14.36, "gender": "F"},
    {"name": "Tom Brown", "dept": "IT", "hourly": 12.56, "gender": "M"},
    {"name": "Amara Okafor", "dept": "IT", "hourly": 21.03, "gender": "F"},
    {"name": "Ben Walker", "dept": "Payroll", "hourly": 17.20, "gender": "M"},
    {"name": "Chloe Evans", "dept": "IT", "hourly": 16.41, "gender": "F"},
]
```

(a) Write two functions: `average(numbers)` and `pay_gap_percent(male_hourly, female_hourly)` (as % of men's rate, 1 decimal).

(b) Build a dictionary `rates` whose keys look like `"IT/F"` and whose values are **lists** of hourly rates. The counting loop from exercise 5, but appending instead of adding 1. Print it.

(c) For `"Payroll"` and `"IT"`, print the men's and women's average and the gap.

EXPECTED OUTPUT

```
{'Payroll/F': [15.9, 14.36], 'IT/M': [18.72, 12.56], 'IT/F': [21.03, 16.41], 'Payroll/M':
[17.2]}
Payroll   men £17.20  women £15.13  gap 12.0%
IT        men £15.64  women £18.72  gap -19.7%
```

Hint: A dictionary whose values are lists is a dictionary like any other. `rates[key].append(x)` reads as: find the list for this key, then append to it.

Bug hunt Dave's Leftovers #4

15 min · `bughunt/staff_list_bug.py`

Dave's staff list, last run in 2019. It found the wrong top earner, crashed looking up a department that definitely exists, and reported a headcount of 1 for everyone. There are **three** bugs: one crashes (read the error twice; the clue is in the quotes), one picks the wrong top earner silently, and one makes every count come out as 1.

EXPECTED OUTPUT ONCE FIXED

```
Highest paid: Amara Okafor
IT: Mo Khan
{'Payroll': 1, 'IT': 2, 'Sales': 1, 'Warehouse': 1}
```

Commit each fix separately, with a message explaining what was wrong, like a detective's report.